



Machine Learning

From Raw Data to Real Predictions — Concepts, Workflow & Tools

What We're Covering Today



What Machine Learning actually is (and isn't)



The three types of ML — with real examples



Exploratory Data Analysis (EDA)



Data Preprocessing — encoding & scaling



Feature Selection — Filter, Wrapper, Embedded



AI vs Machine Learning vs Deep Learning



The end-to-end ML project pipeline



Data Cleaning — fixing what's broken



Feature Engineering — building better inputs



Common pitfalls: overfitting & train/test splits



PART 1

What Is Machine Learning?

The idea that changed how software gets written



What Is Machine Learning?

- Machine Learning (ML) is a branch of AI where a system learns patterns from data instead of following hand-written rules.
- You show it many examples — it works out the pattern connecting inputs to outputs on its own.
- The more relevant data it sees, the better its pattern-matching (usually) gets.
- Formally: a program that improves its performance on a task through experience (data), without being explicitly reprogrammed.

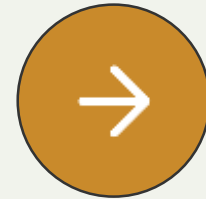


**Learns from
data, not
hard-coded rules**



Traditional Programming vs. Machine Learning

- Traditional programming: you write the rules yourself — Rules + Data go in, Answers come out.
- Machine Learning flips this — Data + Answers (examples) go in, and the machine works out the Rules (the model).
- This flip matters when the rules are too complex, too fuzzy, or unknown to write by hand — like recognizing a face or predicting a price.
- Once trained, that learned 'rule set' (the model) can be applied to brand-new, unseen data.



**Data + Answers
→ Rules
(the model)**



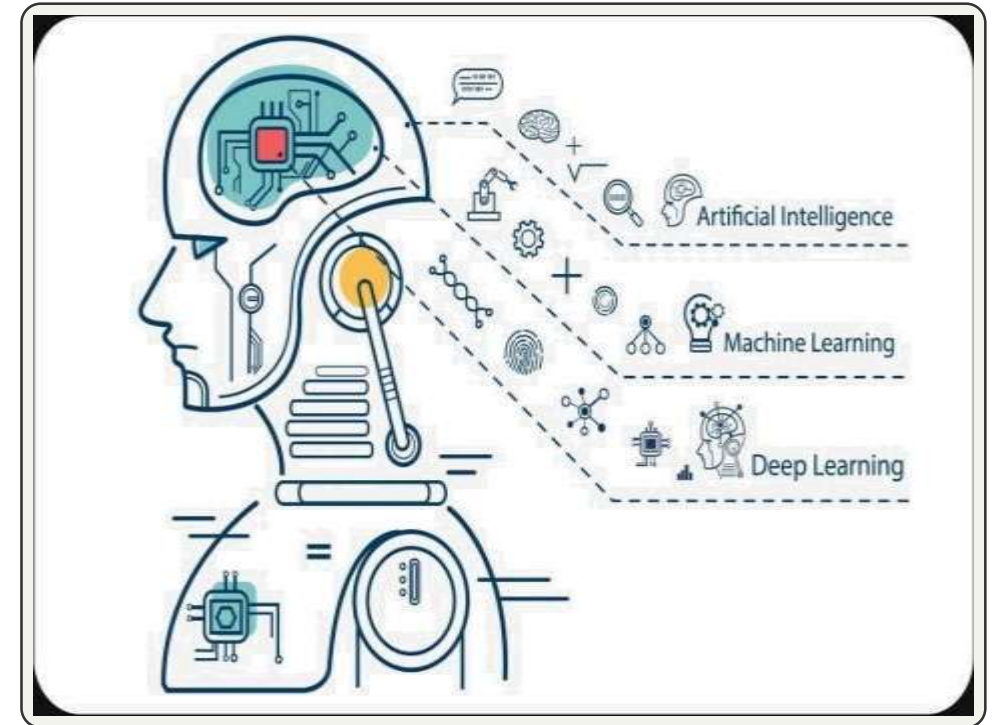
Machine Learning in Everyday Life

- YouTube & Netflix recommendations — learn from your watch history to predict what you'll watch next.
- Spam detection in Gmail — learns the patterns common to spam emails and filters them out.
- Voice assistants (Siri, Alexa, Google Assistant) — learn to understand how you speak.
- Self-driving cars — learn to recognize stop signs, pedestrians, lanes, and other vehicles.
- Banking & fraud detection — learn what a normal transaction looks like, and flag what doesn't.



Real Example: Face Unlock

- Your phone's camera captures thousands of tiny facial measurements — distances between eyes, nose shape, jawline, and more.
- During setup, the model learns the unique pattern that is your face.
- Every time you unlock your phone, it compares the live scan against that learned pattern — in a fraction of a second.
- It even adapts over time to gradual changes, like glasses or a new haircut.



Biometric ML: pattern recognition applied to facial geometry



Why Learn ML Now?

- Today, Machine Learning powers everything from the apps on your phone to the systems running hospitals, banks, and space research.
- It's the engine behind search engines, recommendation systems, medical diagnosis tools, and generative AI.
- Pakistan's tech industry is increasingly hiring for ML/data roles — in fintech, health-tech, and e-commerce.
- Learning ML today means learning the language the next decade of software will be written in.



**The language
of the future
of software**



PART 2

AI vs ML vs DL

Three related terms people mix up constantly



Artificial Intelligence — The Big Umbrella

- AI is the broadest term: the science of building machines that can perform tasks that normally require human intelligence.
- That includes reasoning, understanding language, recognizing images, planning, and decision-making.
- AI is a goal, not one specific technique — it covers everything from simple if/else rule engines to today's deep neural networks.
- Machine Learning is simply the most successful modern approach to achieving AI.



**Machines that
act & reason
like humans**



Machine Learning — A Subset of AI

- ML is one specific way to achieve AI: instead of programming intelligence directly, you let the machine learn it from data.
- It covers a huge family of algorithms — from simple Linear Regression to complex ensemble models.
- Every ML system is AI, but not every AI system uses ML (a chess engine with hand-coded rules is AI, not ML).
- ML systems generally improve as they're exposed to more (good) data over time.



**Learns from
data, gets
better over time**



Deep Learning — A Subset of Machine Learning

- Deep Learning (DL) is a specialized branch of ML built on artificial neural networks with many stacked layers — 'deep' refers to that layer count.
- These networks are loosely inspired by how neurons connect in the human brain.
- DL shines at unstructured data — images, audio, and text — powering things like face recognition, computer vision, and speech-to-text.
- It typically needs much more data and computing power than classic ML algorithms.



**Neural networks,
many layers deep**



AI vs ML vs DL — Side by Side

AI

- The broad goal: machines that think or act intelligently
- Includes rule-based systems, search, planning, ML
- Example: a chess engine, a rule-based chatbot

ML

- A way to reach AI by learning from data
- Needs labeled or unlabeled datasets to train on
- Example: predicting house prices from past sales

DL

- A subset of ML using deep neural networks
- Best for images, audio, video, and text
- Example: face recognition, self-driving perception



PART 3

The Three Types of Machine Learning

Supervised, Unsupervised & Reinforcement Learning

NEEDS LABELS



Supervised Learning

Like a student learning with a teacher — the machine is given both the input data and the correct answer, and it learns to map one to the other.

Example: predicting loan approval from income & credit score



Supervised Learning — A Worked Example

- Say we want to predict whether a bank should approve a loan.
- Income and Credit Score are the input variables (features) — what we already know about an applicant.
- Loan Approved (Yes/No) is the output variable (target/label) — what we want to predict.
- We train the model on historical rows where both the inputs and the correct output are already known.
- Once trained, the model can predict the output for a brand-new applicant it has never seen before.



**Input →
model →
predicted output**



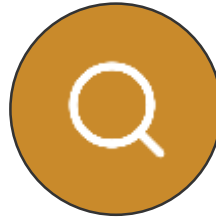
Common Supervised Learning Algorithms

- Linear Regression — predicts a continuous number (e.g. house price).
- Logistic Regression — predicts a yes/no category despite the name 'regression'.
- Decision Trees & Random Forests — split data on questions to reach a decision.
- K-Nearest Neighbors (KNN) — classifies based on the closest labeled examples.
- Support Vector Machines (SVM) — finds the best boundary that separates classes.
- Naive Bayes — a fast probability-based classifier, popular for spam filtering.



**Two flavors:
Regression
(numbers) &
Classification
(categories)**

NO LABELS



Unsupervised Learning

The machine only gets data — no correct answers. It's left to find structure on its own: grouping similar items, spotting outliers, and simplifying complexity.

Example: grouping website visitors into customer segments



Unsupervised Learning — Algorithms & Uses

- Clustering (e.g. K-Means, Hierarchical Clustering) — groups similar data points together, like customer segments.
- Dimensionality Reduction (e.g. PCA) — compresses many features into fewer, more informative ones.
- Anomaly Detection — flags unusual data points, useful for fraud or defect detection.
- Association Rule Learning — finds relationships between items, like 'customers who buy X also buy Y'.
- Real example: Google's search-result clustering and 'customers also bought' recommendations both lean on unsupervised techniques.



**Finds hidden
structure —
no teacher
required**

TRIAL & ERROR



Reinforcement Learning

Like training a dog — you don't explain every rule. You reward good actions and discourage bad ones, and the agent learns the best strategy through trial and error.

Example: game-playing AI, robotics, self-driving decision-making



The 3 Types of ML at a Glance



Supervised

Data + correct answers
Predicts outputs
for new inputs



Unsupervised

Data only
Finds patterns,
groups & outliers



Reinforcement

Rewards & penalties
Learns the best
action by trial



PART 4

The Machine Learning Workflow

How a raw dataset becomes a working model



The 11 Steps of a Machine Learning Project

Every serious ML project — from a student notebook to a production system — follows roughly this path

1

Problem Definition — what exactly are we predicting, and why?

2

Data Collection — gather relevant, sufficient, representative data

3

Exploratory Data Analysis (EDA) — understand the data first

4

Data Preprocessing / Cleaning — fix nulls, duplicates, errors

5

Feature Selection & Engineering — choose and craft the best inputs

6

Split the Dataset — separate training data from test data

7

Model Selection — pick an algorithm suited to the problem

8

Model Training — let the algorithm learn from the training data

9

Model Evaluation — measure performance on unseen data

10

Hyperparameter Tuning — fine-tune the model's settings

11

Model Testing / Validation — confirm it generalizes before deployment



What Is EDA?

- EDA stands for Exploratory Data Analysis — the process of analyzing, visualizing, and understanding your data before building any model.
- Think of it as detective work: you're getting curious about your data before doing any fancy transformations.
- It's almost always Step 3 in the pipeline — right after you collect data, before you touch it.
- You can't clean or engineer what you don't first understand.



**Look before
you touch
the data**



Why EDA Matters

- You can't clean or preprocess what you don't understand — EDA comes first.
- It helps you identify mistakes, biases, or limitations hiding in the data.
- It guides the direction of your data cleaning and feature engineering decisions.
- It gives your audience or stakeholders a story or overview of what the data is telling you, before any model exists.



**Better EDA →
better decisions
downstream**



Core EDA Techniques

- Viewing the data — `.head()`, `.tail()`, `.shape`, `.info()` to see structure and types.
- Summary statistics — mean, median, mode, std, min, max, quartiles via `.describe()`.
- Value counts — how many unique values in a column, great for categorical data.
- Missing value analysis — where are the gaps, and what % of data is missing?
- Visualizations — histograms (distribution), boxplots (outliers & spread), correlation heatmaps (relationships), scatter plots (bivariate patterns).



**Numbers +
visuals =
full picture**



PART 5

Data Cleaning & Preprocessing

Turning messy real-world data into model-ready data



Data Cleaning — Fixing What's Broken

- Handle missing values — check which columns have nulls, then drop, impute (mean/median/mode), or predict them.
- Remove duplicates — detect and drop exact duplicate rows that would bias the model.
- Fix data types — convert wrongly-typed columns, e.g. numbers stored as strings or dates stored as text.
- Fix inconsistent categories — 'Male', 'male', 'MALE' should all become one consistent value.
- Detect & handle outliers — using boxplots, IQR, or z-score; remove, cap, or investigate them.
- Fix logic/domain errors — e.g. age = -5, or an impossible BMI value.



**EDA finds
the problem.
Cleaning fixes it.**



Data Preprocessing — Encoding Categorical Data

- Most ML algorithms only understand numbers — categorical text columns need to be converted first.
- Label Encoding — assigns each category a number (0, 1, 2...); good for ordered categories like 'Low, Medium, High'.
- One-Hot Encoding — creates a separate 0/1 column per category; best for unordered categories like region or color.
- Picking the wrong encoding can accidentally imply an order that doesn't exist in the data — a common beginner mistake.



**Turn text
categories
into numbers**



Feature Scaling — Normalization vs. Standardization

Normalization (Min-Max)

- Rescales values into a fixed 0–1 range
- Formula: $(x - \min) / (\max - \min)$
- Good when data has known, bounded limits
- Sensitive to outliers

Standardization (Z-score)

- Rescales to mean = 0, std = 1
- Formula: $(x - \text{mean}) / \text{std}$
- Works well when data is roughly normal
- More robust to outliers than min-max



PART 6

Feature Engineering & Selection

Giving the model the best possible inputs to learn from



What Is Feature Engineering?

- Feature Engineering means creating new features, or transforming existing ones, to expose patterns that ML models can actually learn from.
- Why bother? Because ML models don't understand domain logic on their own — we have to hand them the right signals.
- Common techniques: mathematical combinations (e.g. price per sq. ft.), target-based flags, binning continuous values into ranges, and time-based features (day of week, month, holiday).
- Good feature engineering often improves accuracy more than switching to a fancier algorithm.



**Better inputs
usually beat
a fancier model**



Why Feature Selection Matters

- Feature Selection means picking the most useful features and removing the rest.
- Reduces noise and overfitting — fewer irrelevant features means the model generalizes better.
- Speeds up training — fewer columns means faster computation.
- Improves accuracy and makes the model easier to interpret and explain.
- There are three broad families of methods: Filter, Wrapper, and Embedded.



**Fewer, better
features beat
more, noisy ones**



Feature Selection Methods — Filter, Wrapper, Embedded

Filter Methods

- Pure statistics, model-independent
- Correlation matrix — drop highly correlated features
- Chi-square test — categorical vs categorical
- ANOVA F-test — numerical vs categorical target
- Fast, but ignores feature interactions

Wrapper Methods

- Test feature subsets using the actual model
- Recursive Feature Elimination (RFE)
- Forward selection / backward elimination
- More accurate, but computationally expensive

Embedded Methods

- Selection happens during model training
- Lasso Regression — shrinks weak coefficients to 0
- Tree-based models (Random Forest, XGBoost)
- → built-in feature importance scores



Try It Yourself — Practice Tasks

Pick any small dataset (e.g. Titanic, Iris, or a Kaggle CSV) and work through these

1

Load the dataset and run `.info()`, `.describe()`, and `.head()`

2

Count missing values per column and decide how to handle each

3

Plot a histogram and a boxplot for one numeric column

4

Plot a correlation heatmap across all numeric columns

5

One-hot encode at least one categorical column

6

Scale a numeric column using Min-Max normalization

7

Split the data using `train_test_split` (e.g. 80/20)

8

Train one simple model (e.g. Logistic Regression) with scikit-learn



RECAP

**Machine Learning is the art of turning data into decisions —
but a model is only ever as good as the data pipeline behind it.**

Next session: hands-on EDA & preprocessing with pandas and scikit-learn